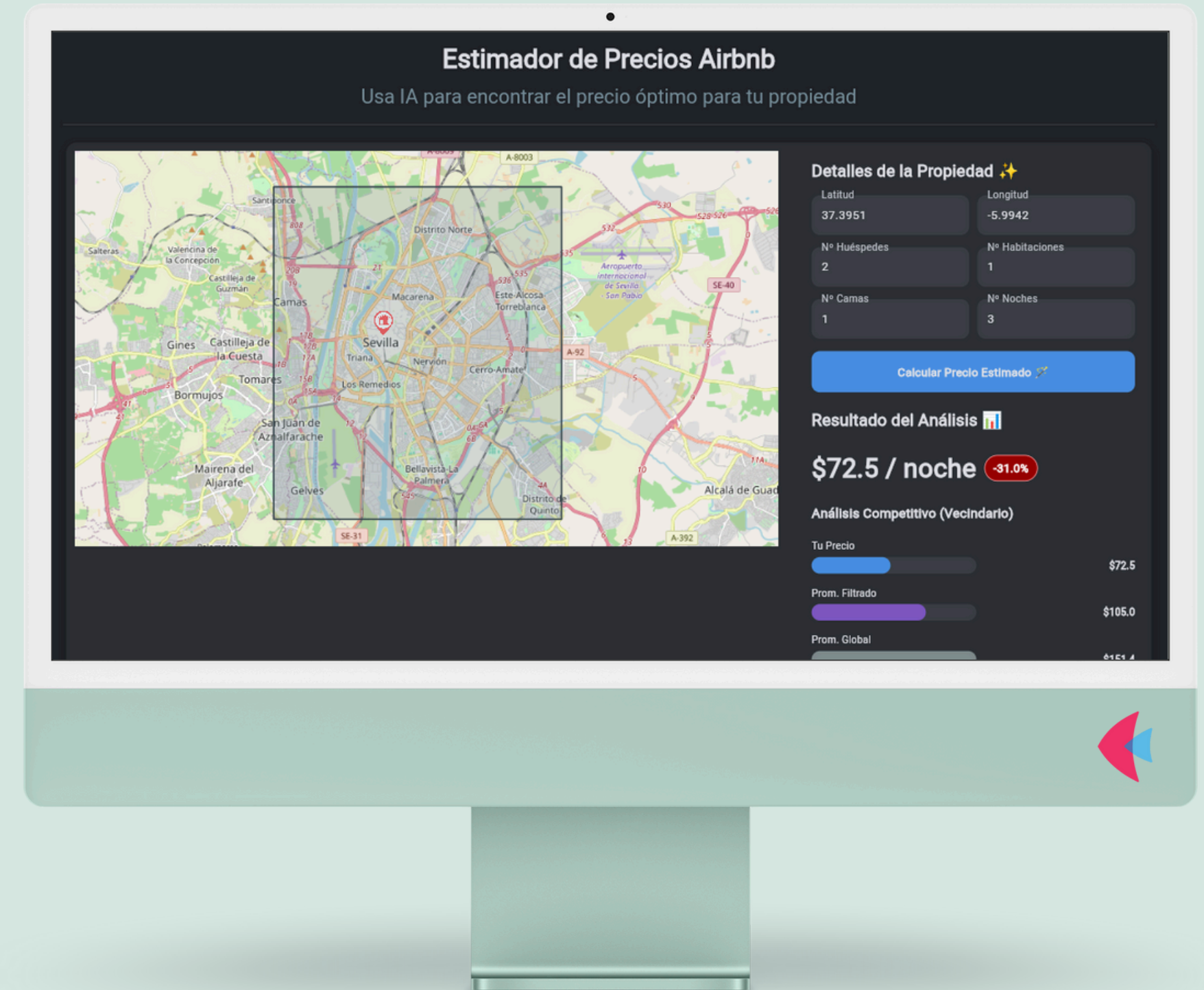


Más allá del modelo:

Presenta tus proyectos Python como aplicaciones interactivas con Flet

Mayumy Carrasco Huaccha

<https://airbnb-pricing-frontend-flet.onrender.com/>





Sobre mí



Mayumy Carrasco Huaccha

Analista de Datos en Ciberseguridad para Telefónica
(a través de VASS).

Me encanta el mundo de los datos ❤️🔥 y, sobre todo,
automatizar procesos con Python 🐍 para darles
vida.

Licenciada en Computación Científica – Perú
Master en BigData y Analytic – España



MayumyCH



heydy-mayumy-carrasco-huaccha



CONTEXTO



Kiomi

**"Una vez que tengo un modelo que funciona...
¿ahora qué?"**



PROBLEMÁTICA



El Desafío de la "Caja Negra"

¿Cómo consigo que mis jefes **confíen y entiendan el modelo** si no pueden interactuar con él?

Dandoles el poder de **simular escenarios** y **ver el impacto en tiempo real**, no un informe estático.



El Desafío de la "Brecha con el Producto"

¿Cómo demuestro el **potencial real** del modelo como un producto usable (web, escritorio, móvil)?

Entregando un **Producto Mínimo Viable (MVP)** que enseñe el valor, no solo el análisis.



BUSQUEDA DE LA SOLUCION



STREAMLIT

Transforma scripts en dashboards web interactivos

Destaca por:

Velocidad y Simplicidad Mágica

Ideal para:

Dashboards de datos, prototipos de ML y visualizar datos (WEB).



FLET

Crea aplicaciones multiplataforma con solo Python.

Destaca por:

Multiplataforma Real

Ideal para:

MVPs, herramientas internas, Apps (Web, Escritorio, Móvil).



DASH

Framework potente para aplicaciones analíticas complejas.

Destaca por:

Potencia y Flexibilidad

Ideal para:

Dashboards de BI, apps científicas complejas (WEB)



Flet, tu Frontend en Puro Python

Construir interfaces de usuario interactivas y multiplataforma sin salir del ecosistema de Python.

01

UI MODERNA CON FLUTTER

Aprovecha el poder del **motor de UI de Google** para crear interfaces fluidas y atractivas sin escribir una línea de Dart.

02

100% PYTHON

La lógica, la interfaz de usuario y la gestión de estado, todo en el mismo lenguaje que ya amas.

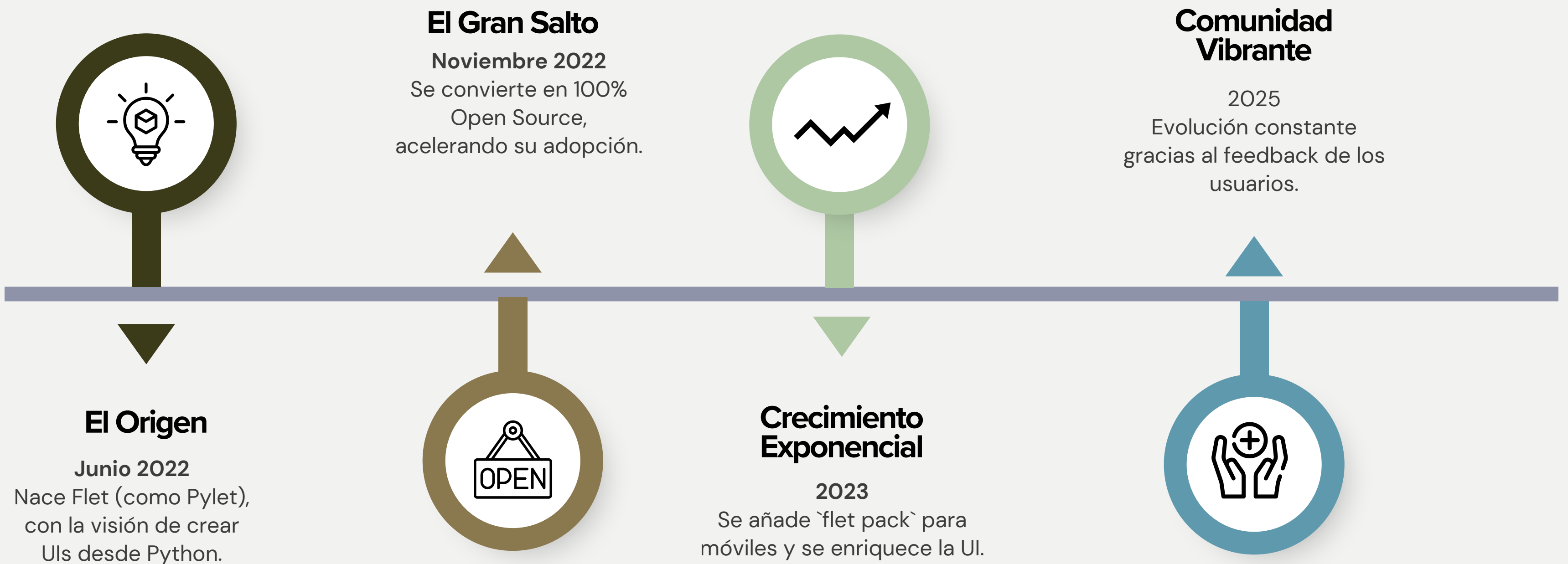
03

CURVA DE APRENDIZAJE SUAVE

Con componentes intuitivos, puedes pasar de una idea a un prototipo funcional en cuestión de horas



¿Por Qué Apostar por Flet?

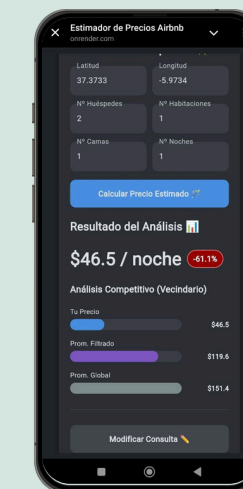
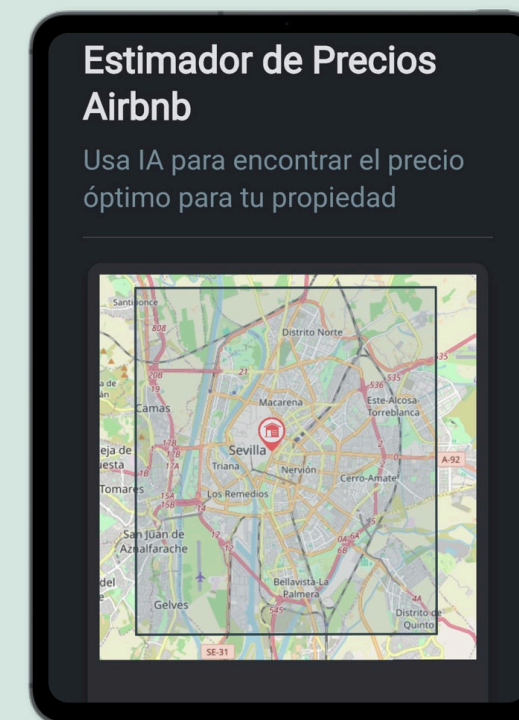
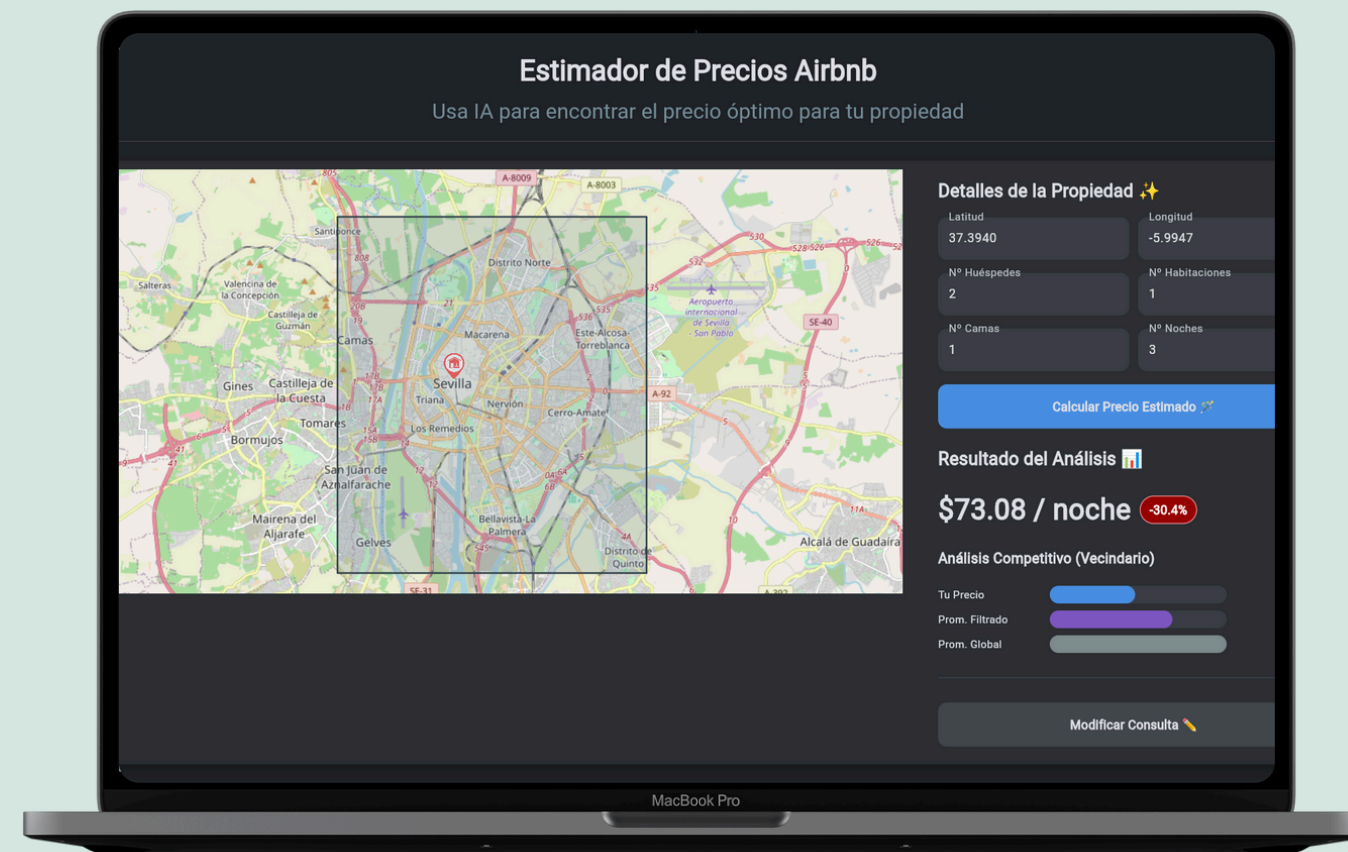


Feodor Fitzner



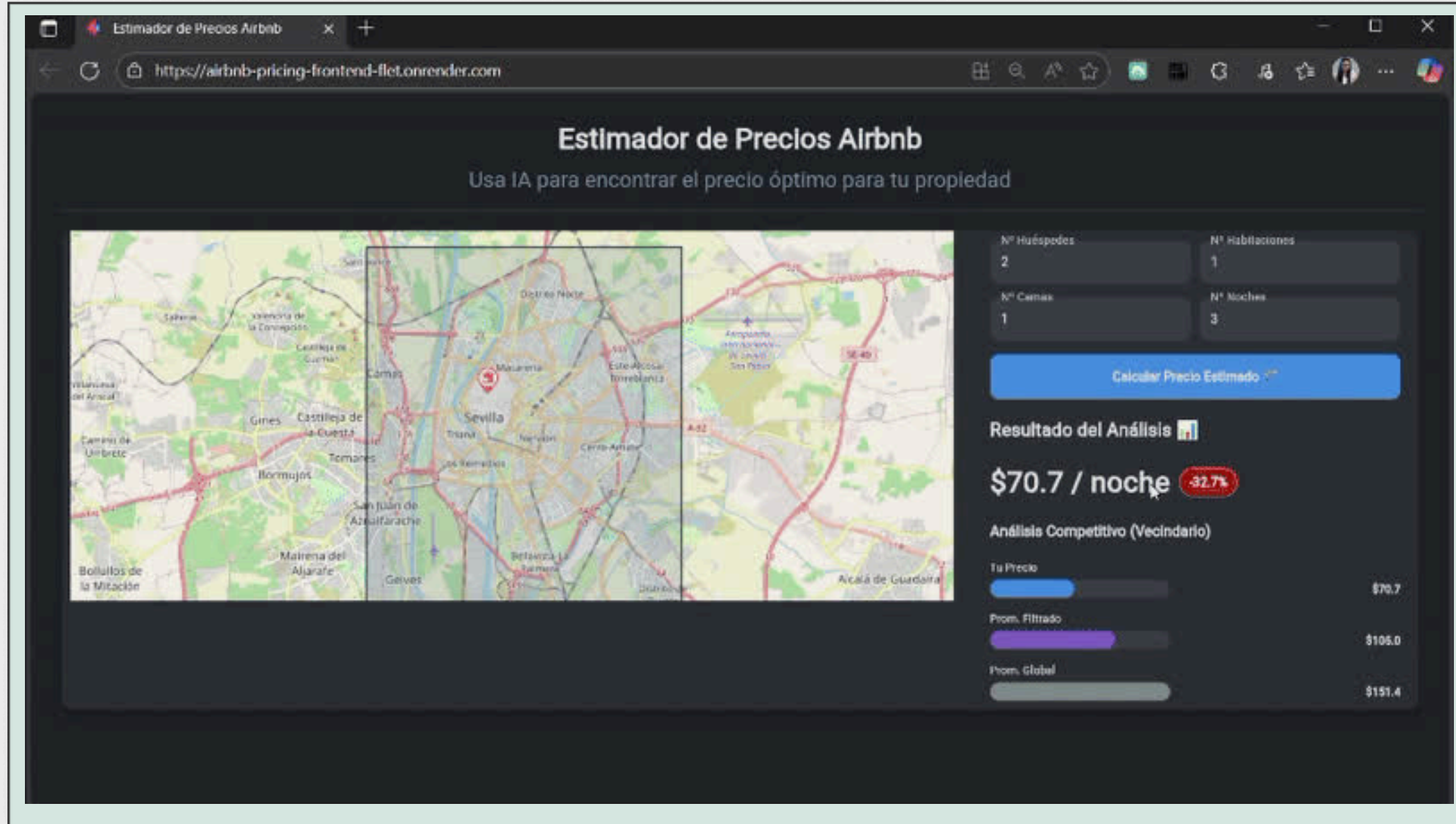
Flet Multiplataforma Real: Un código para gobernarlos a todos

El mismo código Flet que crea tu aplicación web, también genera aplicaciones nativas para escritorio y móvil.





Demo Interactiva



Estimador de Precios Airbnb

Arquitectura del Frontend



```
Airbnb-Pricing-frontend-flet/
├── assets/                # Recursos estáticos (Iconos, imágenes y otros)
│   └── marcadorMapa.png
├── components/            # (Moléculas) Componentes de UI reutilizables.
│   ├── input_form.py     # El formulario para introducir datos.
│   ├── map_view.py       # El widget del mapa interactivo.
│   └── result_panel.py   # El panel que muestra los resultados.
├── services/             # Lógica para comunicarse con servicios externos.
│   └── api_client.py      # Encapsula todas las llamadas a la API del modelo de ML.
├── views/                # (Organismos) Ensambla componentes en vistas completas.
│   └── main_view.py       # La vista principal que organiza toda la pantalla.
├── main.py               # Punto de entrada de la aplicación (Configura y lanza Flet)
├── .dockerignore         # Archivos a ignorar por Docker.
├── Dockerfile            # Define el entorno para la imagen de Docker.
├── requirements.txt      # Lista de dependencias de Python (flet, httpx, etc.).
└── .env.example          # Plantilla para variables de entorno.
```



Orquestador

main.py

Archivo configura la ventana, carga la vista principal que tú creaste y la lanza como una aplicación web.

```
def main(page: ft.Page):
    # --- Configuración de la Página ---
    page.title = "Estimador de Precios Airbnb"
    page.theme_mode = ft.ThemeMode.DARK
    page.bgcolor = "#202328"
    # ...
    # ... (configuraciones de alineación, scroll, etc.)
    # ...

    # --- Cargar la vista principal ---
    main_view = MainView(page)
    page.add(main_view)
    page.update()

if __name__ == "__main__":
    port = int(os.environ.get("PORT", 8550))
    ft.app(
        target=main,
        port=port,
        host="0.0.0.0",
        assets_dir="assets",
        view=ft.AppView.WEB_BROWSER
    )
```



Organismo

views/main_view.py

```
class MainView(ft.Container):
    def __init__(self, page: ft.Page):
        super().__init__(padding=ft.padding.all(20))
        self.page = page

    # --- Instancias de los componentes ---
    self.map_view = MapView(on_map_click=self._handle_map_click)
    self.input_form = InputForm(on_submit=self._handle_submit)
    self.result_panel = ResultPanel(on_edit=self._handle_edit_request)

    # --- Layout Principal Responsivo ---
    self.main_layout = ft.ResponsiveRow(
        controls=[
            # Contenedor del Mapa
            ft.Container(
                content=self.map_view,
                col={"xs": 12, "md": 6, "lg": 6},
            ),
            # Contenedor del Formulario y Resultados
            ft.Container(
                content=ft.Column([self.input_form, self.result_container]),
                col={"xs": 12, "md": 6, "lg": 6},
            ),
        ],
    )
    # ... el resto del layout ...
    return go(f, seed, [])
}
```

- Instancio mis componentes (MapView, InputForm) como si fueran objetos normales.
- Uso ResponsiveRow de Flet para que mi aplicación se adapte automáticamente a móvil y escritorio



Organismo

views/main_view.py

```
async def _handle_submit(self, e: ft.ControlEvent):
    # 1. Muestra un indicador de progreso
    self.result_container.visible = True
    self.page.update()

    # 2. Recoge los datos del formulario
    form_data = self.input_form.get_data()

    # 3. Llama a la API de forma concurrente con asyncio.gather
    api_response, average_response = await asyncio.gather(
        api_client.get_price_suggestion(form_data),
        api_client.get_average_price(form_data)
    )

    # 4. Procesa la respuesta y actualiza la UI
    if "error" in api_response:
        # ... manejar error ...
    else:
        self.result_container.content = self.result_panel
        self.result_panel.update_data(api_response, average_response)

    self.page.update()
```

- Al pulsar 'Calcular', no bloqueo la app, muestro un ProgressRing y llamo a page.update().
- Uso asyncio.gather para lanzar dos peticiones a la API al mismo tiempo, lo cual es muy eficiente.
- Una vez que obtengo la respuesta, actualizo el panel de resultados y vuelvo a llamar a page.update()

NEXT



Capa de Servicio

services/api_client.py

- Uso httpx, una librería moderna que soporta async/await de forma nativa.
- La comunicación con el backend está aislada en su propio módulo (services)
- El try...except robusto me permite manejar errores de conexión o de la API

```
import httpx
import os

PRICE_API_URL = os.environ.get("PRICE_API_URL", "http://127.0.0.1:8000/api/v1/predict")

async def get_price_suggestion(data: dict) -> dict:
    """
    Realiza una llamada asíncrona a la API de predicción de precios.
    """
    async with httpx.AsyncClient() as client:
        try:
            response = await client.post(PRICE_API_URL, json=data, timeout=20.0)
            response.raise_for_status() # Lanza excepción para errores HTTP
            return response.json()
        except httpx.ConnectError:
            return {"error": "No se pudo conectar a la API."}
        except Exception as e:
            return {"error": f"Ocurrió un error inesperado: {str(e)}"}
```




Molecula

components/map_view.py

```
class MapView(ft.Container):
    def __init__(self, on_map_click: Callable):
        super().__init__(expand=True)
        # 1. Se guarda la función que llega desde el padre (MainView)
        self.on_map_click = on_map_click

        self.marker_layer = MarkerLayer(markers=[])

        self.map_control = Map(
            layers=[
                TileLayer(...), # El mapa de OpenStreetMap
                self.marker_layer # La capa para marcadores
            ],
            ...
            # 2. Se conecta el evento de clic del mapa
            on_tap=self._handle_map_click,
        )
        self.content = self.map_control

    def _handle_map_click(self, e: ft.ControlEvent):
        # 3. Pone un nuevo marcador en el mapa
        self.marker_layer.markers.clear()
        self.marker_layer.markers.append(
            Marker(...)
        )

        # 4. Llama a la función del padre y le pasa los datos
        self.on_map_click(lat, lon)
        self.update()
```

- Recepción de Tarea (Padre a Hijo):
Recibe la función del MainView al crearse.
- Callback (Hijo a Padre):
Al hacer clic, actualiza su propio estado (pone un marcador) y devuelve la latitud y longitud.



Del Código a una App Real: El Empaquetado

Camino A: App de Escritorio

Paso 1: El Único Requisito

Asegúrate de tener PyInstaller. Flet te ayudará a instalarlo si no lo tienes.

```
pip install pyinstaller
```

Paso 2: El Comando Mágico

En tu terminal, simplemente ejecuta:

```
flet build exe main.py
```

Paso 3: ¡Aplicación Lista!

Busca en la carpeta 'dist'. Tu archivo '.exe' estará allí, listo para compartir.

Camino B: App Móvil (Android)

Paso 1: Preparación (Una sola vez)

Necesitas el entorno de Android: Flutter SDK, Android Studio, y JDK.

Paso 2: Chequeo de Salud

El comando 'flutter doctor' te guiará para asegurar que todo esté configurado.

```
flutter doctor
```

Paso 3: El Comando Mágico (Android)

Una vez configurado, el comando es casi idéntico

```
flet build apk main.py
```



LECCIONES APRENDIDAS

PRO

Velocidad de desarrollo increíble.
Pasar de la idea a un MVP funcional es cuestión de horas.

1

RETO

3

Empaquetado para móvil.
La función `flet build` está madurando y puede requerir configuración adicional.

PRO

Un stack 100% Python.
Mantiene la consistencia y reduce la carga cognitiva.

2

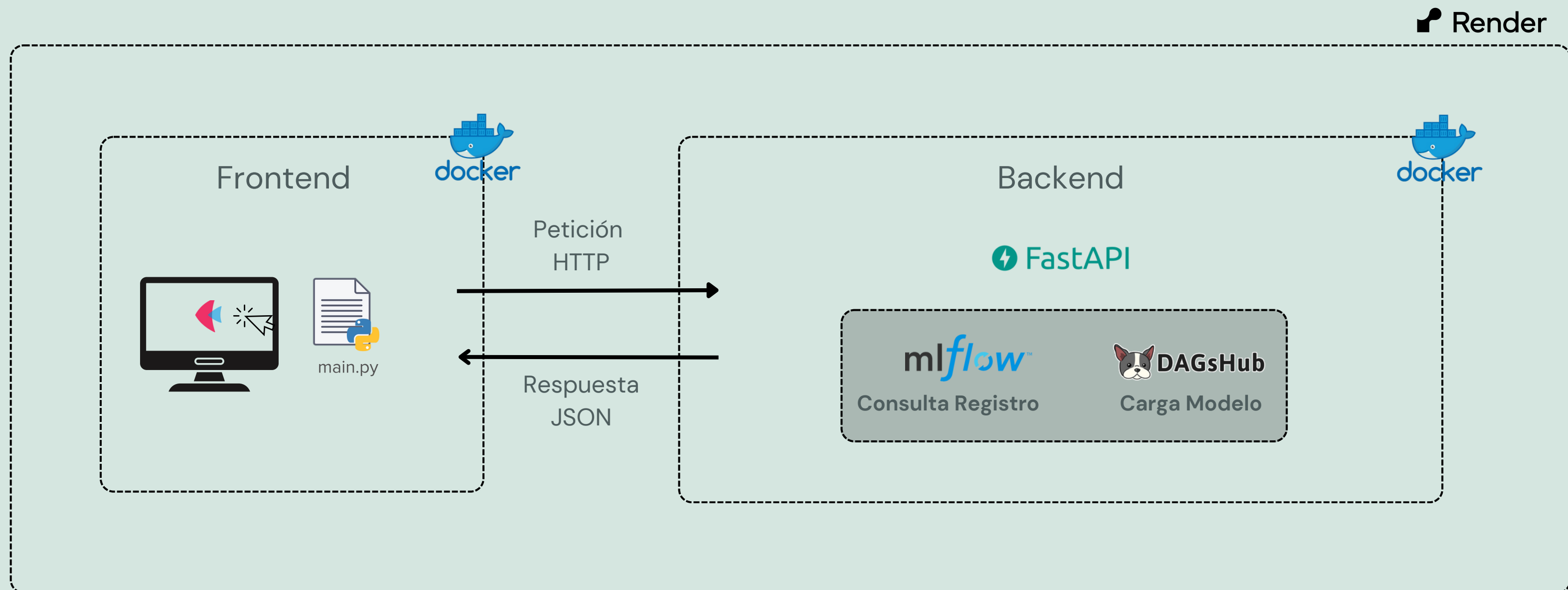
RETO

4

Comunidad joven.
Aunque activa y en crecimiento, no tiene tantos recursos como frameworks más establecidos.



Arquitectura del Proyecto: Flujo en Producción



Gracias Totales !



 MayumyCH

 heydy-mayumy-carrasco-huaccha

 linktr.ee/mayumych

